RALLYSTATS · DOCUMENTO INTERNO

Funcionalidades pendientes: análisis de implementación

Las 10 funcionalidades del roadmap premium que todavía no están hechas, ordenadas de la más fácil a la más difícil según el estado actual del código, con lo que se reusa, lo que falta construir, los riesgos y un boceto de cómo se vería cada una.

Basado en una revisión del código al 3 de octubre de 2026: `volley_stats_app` en `cc57d27` y `volley_stats_app_backend` en `f7b3c31`.
Complementa al *Roadmap de funcionalidades premium* (30/08/2026).
Los bocetos son ilustrativos (muestran ubicación y flujo, no un diseño final). Es un insumo de decisión, no un plan cerrado.

> **Sección 3 revisada el 14 de octubre de 2026:** suma los datos locales por cuenta y el requisito de no perder ningún partido ya archivado. Reemplaza a la *Spec de sincronización de partidos a Firestore* (25/09/2026), que queda obsoleta. Las funcionalidades 1, 2 y 4 (Tanda 1) quedan marcadas como implementadas el 04/10/2026, en la versión 1.2.0.

Contenido

A Qué cambió desde el roadmap del 30/08

B Tabla comparativa y esfuerzo relativo

1 Notas de scouting por partido · implementada el 04/10 — Muy bajo

2 Exportar estadísticas a CSV (Excel) · implementada el 04/10 — Bajo

3 Sincronización de partidos a Firestore · revisada el 14/10 — Medio + validación alta

4 Comparador de dos partidos · implementada el 04/10 — Bajo-medio

5 Panel de temporada — Medio-bajo

6 Fixture con autocompletado — Medio

7 Animar una jugada de pizarra — Medio

8 Backup en la nube (equipos y jugadas) — Alto

9 Scouting fase 2 y carga precisa (Etapa 3) — Alto

10 Chatbot conversacional (IA) — Muy alto

C Orden sugerido de implementación

A. Qué cambió desde el roadmap del 30/08

Varias funcionalidades del roadmap ya se implementaron desde entonces, algunas de una forma más simple que la que se proponía. No entran en la comparación de este documento:

Funcionalidad del roadmap	Estado	Cómo quedó
§2 Scouting de rival, fase 1	Hecho	`scouting_engine.dart` (`buildReport`) + `rival_scouting_screen.dart`, con botón propio en el Home. **Hoy no está gateado a premium**: falta decidir si queda así.
§3 Estadísticas por rotación	Hecho	`StatsEngine.computeRotations` reproduce la rotación de cada rally con `_replayRotations`. No hizo falta agregar `rotationSlot` a `RallyEvent` ni tocar `MatchController`, y funciona también con partidos viejos.
§8 Gráfico de progresión del marcador	Hecho	`computeTimelines` + `score_timeline_chart.dart`, con gráficos propios en `lib/widgets/charts/` (sin sumar `fl_chart`).
§8 Reportes PDF personalizables	Parcial	Con premium ya se eligen qué gráficos y mapas van al PDF (`VisualStatsPreferences`). Falta elegir secciones o columnas de la tabla de jugadores.

Las Etapas 1 y 2 de la estadística visual (pestañas "Mapas" y "Gráficos") están implementadas desde el 02/10/2026, todavía sin publicar. La Etapa 3 quedó pospuesta y aparece acá como parte de la funcionalidad 9.

B. Tabla comparativa

Ordenada de la más fácil a la más difícil según la revisión del 03/10 (el 3 se re-estimó después, ver la nota al pie; 1, 2 y 4 ya están hechas). "Carga en vivo" indica si toca `MatchController` o la captura de jugadas, que es la parte más delicada de la app; "Backend" indica si requiere cambios en el repo `volley_stats_app_backend`.

#	Funcionalidad	Qué ya existe y se reusa	Qué falta construir	Carga en vivo / Backend	Esfuerzo
1	**Notas de scouting por partido**	El patrón de campo opcional con migración `null` en `VolleyMatch`	Campo `notes` + `toJson` / `fromJson` + un `TextField` en el resumen	No / No	Muy bajo Hecho 04/10
2	**Exportar a CSV**	`match_export_service.dart` (share en mobile, `FilePicker` en desktop) y `MatchStats.orderedRows`	Un servicio nuevo de unas 80 líneas + un botón al lado del PDF	No / No	Bajo Hecho 04/10
3	**Sync de partidos a Firestore**	`cloud_firestore` ya está en el pubspec; `account_devices` sirve de modelo para la regla por `uid`; Exportar/Importar queda como red de seguridad	Etapa 1: Hive por cuenta sin mover datos. Etapa 2: servicio de sync con subida inicial, papelera, interruptor con aviso y política de privacidad nueva	No / Sí, solo la regla	Medio validación alta
4	**Comparador de 2 partidos**	`StatsEngine.compute()` tal cual	Pantalla de selección + vista en dos columnas. Sacar `_StatsTable` a un widget compartido	No / No	Bajo-medio Hecho 04/10
5	**Panel de temporada**	`scouting_engine.dart` ya resuelve la agregación de varios partidos	Agregar por `playerId`, hacer públicos `_accumulate` / `_accumulateRecepcion`, filtros y reusar la tabla del punto 4	No / No	Medio-bajo
6	**Fixture con autocompletado**	`pickMode` de `PlaysArchiveScreen`, el patrón de Hive box y el combo de rival de `NewMatchScreen`	Modelo nuevo, box nueva, 2 pantallas, `fixtureEntryId` en `VolleyMatch` y un tile en el Home	No / No	Medio
7	**Animar jugada de pizarra**	`Play.strokes` ya guarda el orden de los trazos	Animación en `WhiteboardPainter` + controles de reproducción	No / No	Medio
8	**Backup en la nube**	Con el sync del punto 3 los partidos ya quedan cubiertos	Faltan equipos y jugadas, con su política de conflictos y las fotos	No / Sí	Alto (baja si antes se hace el 3)
9	**Scouting fase 2 / Etapa 3**	Mapas y coordenadas de la Etapa 2, `zone_heatmap.dart`	Cargar la zona del rival o el toque preciso en vivo, con campos nuevos en `RallyEvent`	**Sí** / No	Alto
10	**Chatbot IA**	El backend ya tiene estructura de Cloud Functions con secretos (`redeemPromoCode`)	Proxy, armado del contexto, anonimización, límites de costo y UI	No / Sí, función nueva	Muy alto

Esfuerzo relativo



Escala cualitativa, no en días. En el sync (3), lo rayado es la validación: actualizar encima con datos reales, dos dispositivos y modo avión. El 3 subió de esfuerzo en la revisión del 14/10 y conserva su número para no romper referencias.

1. Notas de scouting por partido

Implementada el 04/10/2026 · v1.2.0

Muy bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Que el cuerpo técnico pueda dejar observaciones en texto libre sobre cada partido (cómo saca el rival, qué rotación nos costó, qué reforzar) y volver a leerlas antes del próximo cruce con ese rival.

QUÉ YA EXISTE

- El patrón de campo opcional nuevo: `null` en partidos viejos, como ya se hizo con `targetZone` y `missType` en `RallyEvent`.
- `AppDataController.saveMatch` para persistir. Al ser un partido que ya existe no pasa por el tope de 3.
- La pantalla de scouting, que ya lista los partidos contra cada rival.

QUÉ HAY QUE CONSTRUIR

- `volley_match.dart`: `String? notes` + `toJson` / `fromJson`.
- `match_summary_screen.dart`: tarjeta "Notas" en la pestaña Tabla, con un diálogo de edición multilínea.
- Opcional: mostrar las notas en `rival_scouting_screen.dart` debajo de cada partido, y como sección del PDF.



Boceto. Tarjeta de notas en el resumen del partido, el diálogo de edición y (opcional) las notas en el scouting del rival.

Gating y decisiones

- **¿Gratis o premium?** Por sí sola mueve poco la aguja como argumento de venta. Recomendación: dejar la edición gratis y que las notas se lean dentro del scouting (si el scouting pasa a ser premium, el valor queda del lado pago). Si se prefiere gatear, alcanza con envolver el ✎ en `runIfPremium`.
- Si se implementa el sync (3), las notas viajan solas dentro de `toJson()`; editarlas vuelve a subir el partido porque ya está terminado. No hay que hacer nada extra.

2. Exportar estadísticas a CSV (Excel)

Implementada el 04/10/2026 · v1.2.0

Bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: solo si el usuario lo comparte

Objetivo. Que el cuerpo técnico se lleve la tabla de estadísticas a una planilla propia (Excel, Google Sheets) para cruzarla con sus datos, además del PDF de solo lectura que ya existe.

QUÉ YA EXISTE

- `MatchExportService`: hoja de compartir nativa en Android/iOS y `FilePicker.saveFile` en desktop. El CSV usa exactamente el mismo esqueleto.
- `MatchStats.orderedRows` y las 33 columnas de `_StatsTable` (N°, Jugador, Pts, Err, Saq PP… Rec %, Am, Ro).
- El selector de set del resumen: se exporta lo mismo que se está viendo (`StatsEngine.compute(match, setNumber: …)`).

QUÉ HAY QUE CONSTRUIR

- Nuevo `lib/services/csv_export_service.dart`: arma las filas a mano (sin paquete nuevo), una por jugador + "Total Equipo".
- `match_summary_screen.dart`: un `IconButton` nuevo al lado del PDF, envuelto en `runIfPremium`.
- Más adelante: reutilizar el mismo servicio para el scouting y el panel de temporada.



Boceto. Botón nuevo en la barra del resumen, hoja de compartir nativa y el archivo abierto en Excel, con las mismas columnas que la tabla de la app.

> **Corrección al roadmap: el gating no viene incluido.** El roadmap asumía que alcanza con que la pantalla contenedora esté gateada, pero `MatchSummaryScreen` deja que un usuario free vea un partido (`freeStatsMatchId`). El botón de CSV tiene que chequear premium por su cuenta con `runIfPremium` (`lib/widgets/premium_gate.dart`).

Detalles para que abra bien en Excel en español

- **Separador `;` en vez de `,`.** Excel configurado en español usa `;` como separador de listas; con comas, todo queda amontonado en la columna A. Los nombres que tengan `;` o comillas van entre comillas.
- **BOM UTF-8** (`\uFEFF`) al principio del archivo. Sin él, Excel rompe los acentos y la ñ de los nombres.
- **Porcentajes como enteros** (`67` en la columna "Saq %"), para evitar el problema de coma o punto decimal.

3. Sincronización de partidos a Firestore

Medio · Validación alta · Carga en vivo: no · Backend: regla nueva · Datos salen del dispositivo: sí, si el usuario lo activa

> **Revisada el 14/10/2026.** Reemplaza la versión del 03/10 y deja obsoleta la *Spec de sincronización de partidos a Firestore* (25/09): suma los datos locales por cuenta, la subida del archivo existente, la papelera y el consentimiento, con un requisito que manda sobre todo lo demás: **no se puede perder ningún partido ya archivado**.

Objetivo. Que el archivo de partidos sea por **cuenta** y no por dispositivo: un partido terminado en la tablet aparece en el celular de la misma cuenta. La carga en vivo, el archivo y el scouting siguen leyendo de Hive, 100% offline.

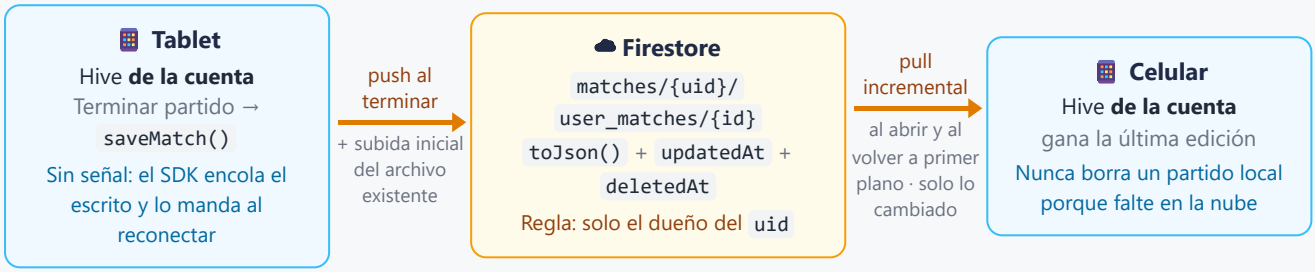


Diagrama. Solo viajan partidos terminados y solo con la nube activada. Un borrado viaja como marca (`deletedAt`) y manda el partido a la papelera de cada dispositivo.

El caso típico: cargar en el teléfono y abrir en otro dispositivo

HOY

No se ve. Todo vive en el Hive del teléfono; para pasarlo hay que usar Exportar/Importar a mano.

CON ESTA FUNCIONALIDAD

Sí se ve, si se cumplen tres condiciones:

1. El partido está **terminado** (los partidos en curso siguen pasando por Exportar/Importar).
2. La cuenta tiene **la nube activada**.
3. El teléfono **tuvo señal antes de cerrar sesión**: la cola offline de Firestore está atada a la cuenta, así que si se cierra sesión antes, el partido sale recién cuando esa cuenta vuelva a entrar en ese teléfono con conexión.

Decisiones

Tema	Decisión	Por qué
Qué se sube	Un interruptor por cuenta, no una elección partido por partido	Un entrenador casi nunca quiere elegir partido por partido, y esa elección genera casos confusos entre dispositivos. Se puede sumar después sin rehacer nada.
Consentimiento	Apagado hasta que el usuario acepta un aviso único	Los planteles llevan fechas de nacimiento, posiblemente de menores. Hasta aceptar, todo sigue 100% local, como promete la política actual.
Free o premium	Para todos (sugerido, a confirmar)	Es el respaldo de lo que el usuario ya tiene; con 3 partidos el costo es mínimo. El valor pago sigue en el archivo ilimitado y los dispositivos simultáneos.
Conflictos	Gana la última edición (`updatedAt`)	En un partido terminado solo se editan las notas.
Borrar	Papelera de 30 días, en todos los dispositivos	Un borrado por error se puede deshacer desde cualquier dispositivo.

Dos etapas, en versiones separadas

ETAPA 1 · DATOS LOCALES POR CUENTA

Se puede publicar sola: arregla un bug actual

Hoy `teams_box`, `matches_box` y `plays_box` son uno por instalación y `signOut` no los toca: otra cuenta en el mismo teléfono ve el archivo de la anterior. Con el sync, eso mezclaría partidos entre cuentas.

- **Los cajones actuales no se mueven:** pasan a ser de la cuenta dueña (`legacy_owner_uid` en `settings_box`), la que tiene la sesión abierta al actualizar.
- Las demás cuentas estrenan `matches_box_<uid>`, etc. `settings_box` sigue siendo del dispositivo.
- Copia JSON de todo antes del primer cambio.
- `loadAll()` pasa a correr después del login.

ETAPA 2 · SINCRONIZACIÓN

Requiere la Etapa 1 y la política de privacidad nueva

- Nuevo `match_sync_service.dart`: push, subida inicial reanudable y pull incremental.
- `updatedAt` y `deletedAt` opcionales en `VolleyMatch` (`null` en partidos viejos).
- Papelera de 30 días con su pantalla, fuera del archivo y del tope de 3.
- Aviso único + switch "Partidos en la nube" en Cuenta.
- `deleteAccount` borra `matches/{uid}`, ofreciendo antes exportar todo.
- Política de privacidad, sección Seguridad de los datos de Play y regla en `firestore.rules`.

Garantías para no perder partidos

Riesgo	Cómo se evita
La migración a cajones por cuenta se corta a la mitad o falla	**No hay migración:** los cajones actuales quedan tal cual y solo se anota su dueño.
Algo sale mal en la primera versión con este cambio	Copia JSON automática de todo el archivo, equipos y jugadas, antes de tocar nada.
Los partidos ya archivados nunca suben (no vuelven a pasar por `saveMatch`)	**Subida inicial** de todo el archivo al activar la nube, reanudable y sin tope de 3.
Un borrado por error se propaga a todos los dispositivos	Papelera de 30 días, restaurable desde cualquiera.
Un error de red o de reglas hace "desaparecer" partidos de Firestore	La bajada nunca borra un partido local porque falte en la nube: solo actúa ante una marca `deletedAt` explícita.
Cerrar sesión, apagar la nube o entrar con otra cuenta	Ninguno toca los datos locales. Apagar la nube pregunta aparte si borrar la copia remota (por defecto, no).
Dos cuentas ya usaron el mismo teléfono antes de esta versión	**Límite aceptado:** sus datos están mezclados y quedan con la cuenta dueña. No se pierden; se pasan con Exportar/Importar.

> **Puntos de la spec del 25/09 que cambian.** Subía solo desde `saveMatch` (el archivo existente nunca llegaba a la nube), borraba de Hive sin avisar a la nube (los partidos borrados reaparecían), leía todo el archivo en cada apertura, aplicaba el tope de 3 al bajar y leía `isPremium` antes de tiempo. La spec daba sus decisiones por cerradas y traía el código casi literal, así que conviene no usarla más: los prompts 3A y 3B la reemplazan.

Cómo probarlo

- **Actualizar encima:** instalar el APK publicado con partidos reales y actualizar sin desinstalar, con sesión abierta, sin sesión y con una segunda cuenta. El archivo tiene que quedar completo.
- **Dos dispositivos:** terminar un partido en uno y verlo en el otro; borrarlo, verlo en la papelera de ambos y restaurarlo.
- **Modo avión:** terminar sin señal, reconectar y verificar que llega solo.
- **Sin tocar datos reales:** Firebase Emulator Suite o una cuenta de prueba descartable.

4. Comparador de dos partidos

Implementada el 04/10/2026 · v1.2.0

Bajo-medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

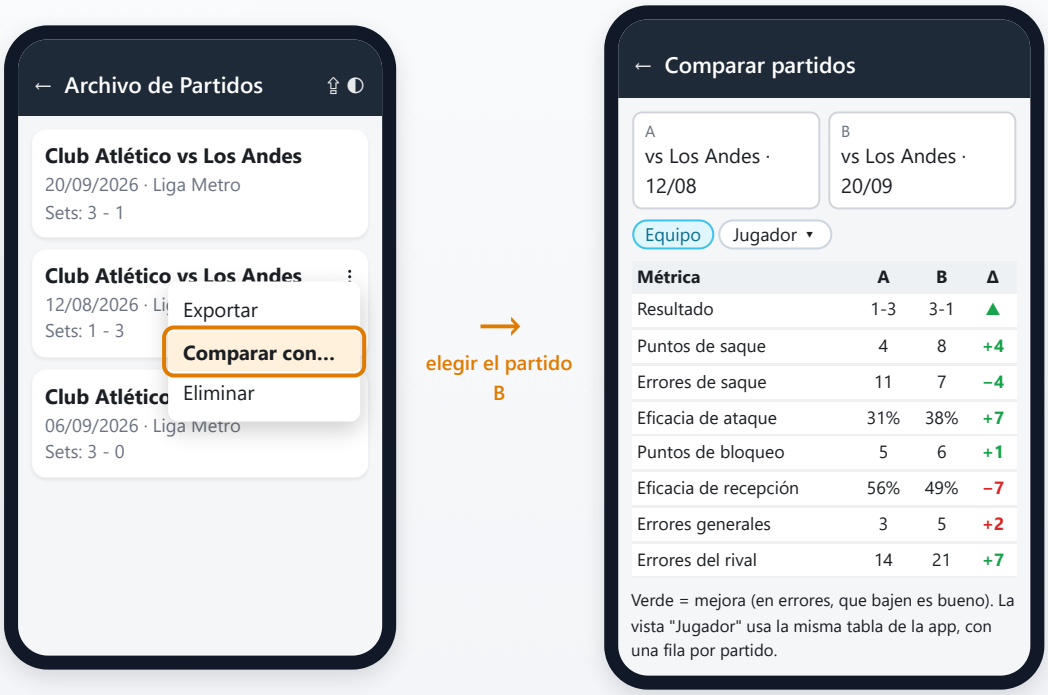
Objetivo. Poner dos partidos lado a lado, por ejemplo la ida y la vuelta contra el mismo rival, o un partido antes y otro después de cambiar el sistema de juego, y ver de un vistazo qué mejoró y qué empeoró.

QUÉ YA EXISTE

- `StatsEngine.compute()` se llama una vez por partido; no hace falta agregación nueva.
- `MatchStats` ya trae totales de equipo y errores y puntos del rival.
- El menú ⋮ de cada partido en el archivo (hoy "Exportar" y "Eliminar").

QUÉ HAY QUE CONSTRUIR

- **Refactor:** sacar `_StatsTable`, `_H` y `_C` de `match_summary_screen.dart` a `matches/widgets/stats_table.dart`, sin cambios visuales. Esa tabla también la usa el Panel de temporada.
- Nueva `matches/match_compare_screen.dart`: dos selectores, una vista "Equipo" y una vista "Jugador".
- Entrada "Comparar con..." en el menú ⋮, que deja preseleccionado el partido A.



Boceto. Entrada desde el menú ⋮ del archivo y comparación de equipo con diferencia coloreada.

Gating y riesgos

- **Premium natural:** `runIfPremium` en "Comparar con...". Un usuario free solo puede ver la estadística de un partido, así que comparar ya excede lo que tiene.
- **Jugadores entre partidos:** `ownRoster` es una copia por partido; el mismo jugador conserva su `Player.id` si los dos partidos se armaron desde el mismo `Team`. Si se cargó un plantel a mano, la vista "Jugador" no puede emparejar (la de equipo sí funciona). Conviene mostrar solo los jugadores presentes en los dos partidos.

5. Panel de temporada

Medio-bajo | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Ver la evolución de cada jugador y del equipo sumando todos los partidos guardados, con filtros por torneo y por rango de fechas. Hoy la estadística existe solo partido por partido.

Por qué ahora es más fácil que en el roadmap: `scouting_engine.dart` ya resuelve el problema de fondo, que es filtrar partidos, recorrerlos y sumar resultados de `StatsEngine` en un reporte. El panel de temporada es lo mismo pero agrupando por jugador propio en vez de por rival.



Diagrama y boceto. El motor de agregación reutiliza todo lo que ya calcula `StatsEngine`; la pantalla reutiliza la tabla del resumen y los gráficos de `lib/widgets/charts/`.

QUÉ HAY QUE CONSTRUIR

- Nuevo `lib/services/season_engine.dart` (calcado de `scouting_engine.dart`).
- Hacer públicos `_accumulate` / `_accumulateRecepcion`, o moverlos como métodos `addFrom()` en `stat_line.dart`, en vez de duplicarlos.
- Nueva `matches/season_stats_screen.dart` con filtros, indicadores y la tabla compartida.
- Entrada: tile "Mi temporada" en el Home con `runIfPremium`.

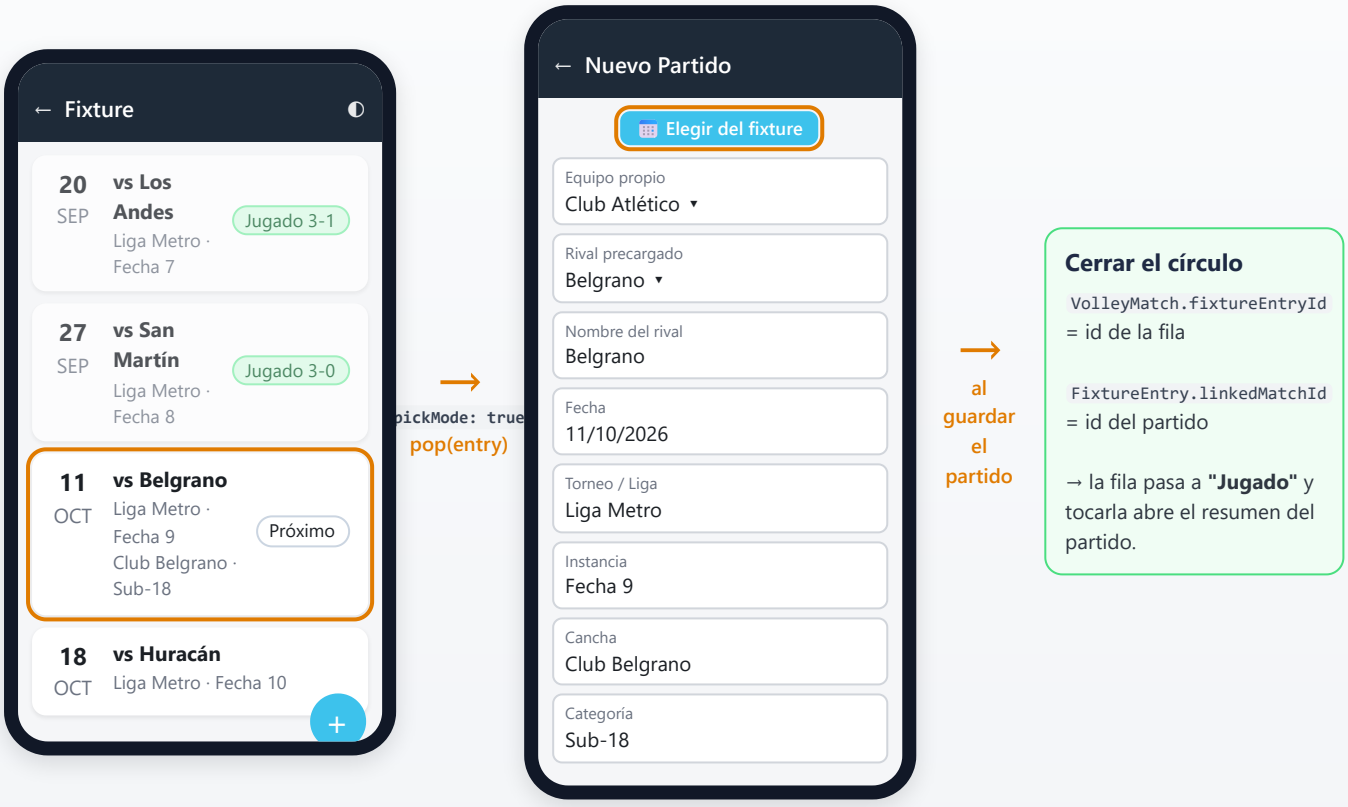
DECISIONES Y RIESGOS

- **Premium natural:** con el tope de 3 partidos, un free casi no tiene temporada para ver.
- Nombre y dorsal: usar los del partido más reciente si cambiaron.
- Partidos sin `ownTeamSourceId` (equipo cargado a mano): agrupar por `ownTeamName` como alternativa.
- Con muchos partidos, calcular fuera del `build` (una vez por cambio de filtro).

6. Fixture con autocompletado

Medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Cargar el calendario del torneo (fecha, rival, instancia, cancha, categoría) y, al crear un partido, elegirlo del fixture para que los datos se completen solos. Además le da al usuario un motivo para abrir la app entre partido y partido.



Boceto. Lista del fixture, autocompletado en Nuevo Partido (campos resaltados) y vínculo entre la fila del fixture y el partido jugado.

QUÉ YA EXISTE

- `PlaysArchiveScreen(pickMode: true)`: el mismo mecanismo de "elegir y devolver con `Navigator.pop`".
- El patrón de Hive box (`teams_box`, `matches_box`, `plays_box`) y de `toJson` / `fromJson` manual.
- `NewMatchScreen` ya autocompleta `_rivalNameCtrl` al elegir un `Team` rival; el fixture es otra fuente para los mismos controles.

QUÉ HAY QUE CONSTRUIR

- `models/fixture_entry.dart`; `fixture_box` en `storage_service.dart`; `fixtureEntries` + alta, baja y edición en `AppDataController`.
- `screens/fixture/fixture_screen.dart` y `fixture_entry_form_screen.dart`.
- `new_match_screen.dart`: botón + `_pickFromFixture()`.
- `fixtureEntryId` opcional en `VolleyMatch` y tile nuevo en el Home.

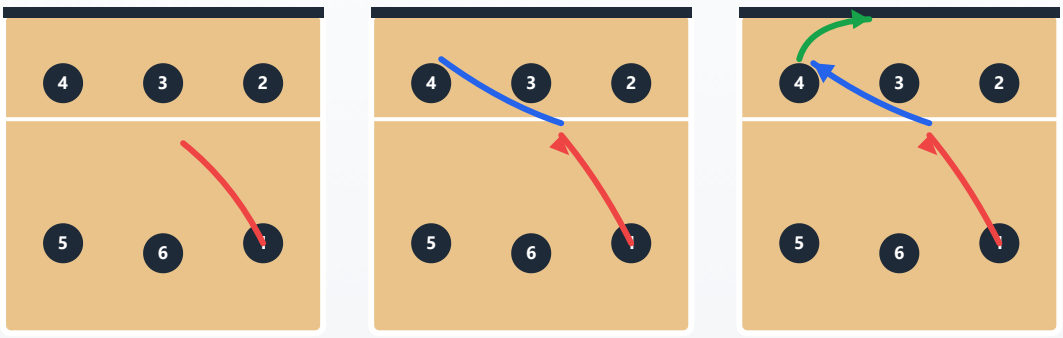
Decisiones pendientes

- **¿Gratis o premium?** Es una función de organización, no de estadística. Opciones: liberada como gancho de uso frecuente, o con un tope en free (por ejemplo, 5 filas) siguiendo el patrón de `MatchArchiveLimitException`.
- **¿Se sincroniza?** Si se hace el sync (3), el fixture quedaría por dispositivo mientras los partidos pasan a ser por cuenta. Conviene decidir si el fixture entra en el sync o en el backup (8).

7. Animar una jugada de pizarra

Medio | Carga en vivo: no | Backend: no | Datos salen del dispositivo: no

Objetivo. Reproducir una jugada guardada dibujándose trazo por trazo, como si el entrenador la estuviera dibujando en ese momento, para explicársela al equipo en el banco o en la práctica.



Boceto. Tres momentos de la reproducción (saque, armado, ataque) y la barra de controles. Las fichas quedan fijas desde el inicio y se animan los trazos en el orden en que se dibujaron.

QUÉ YA EXISTE

- `Play.strokes`: cada `PlayStroke` guarda `pointsX` / `pointsY` en el orden en que se dibujaron, con su color y si termina en flecha.
- `WhiteboardPainter` ya dibuja trazos, flechas y fichas.
- `PlaysArchiveScreen` para elegir la jugada.

QUÉ HAY QUE CONSTRUIR

- `WhiteboardPainter` recibe un `progress` (0 a 1): dibuja los trazos completos hasta N y una parte del trazo N+1. La flecha aparece solo cuando su trazo termina.
- Modo "Reproducir" (pantalla nueva o modo de `whiteboard_screen.dart`) con un `AnimationController`, play/pausa, barra para adelantar o retroceder y velocidad.

Alcance y gating

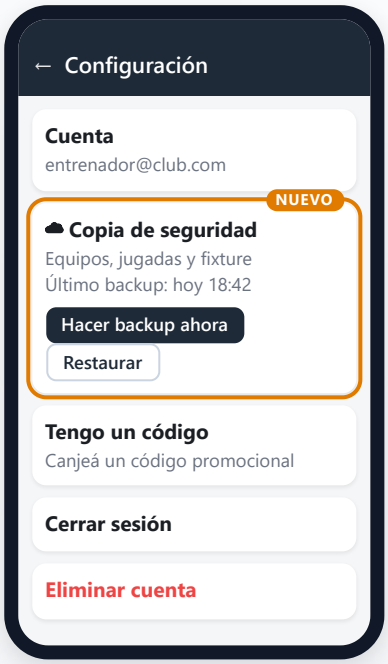
- **Las fichas no se mueven:** `PlacedToken` guarda solo su posición final, no un recorrido. Animar el desplazamiento de jugadores requeriría grabar trayectorias, y eso queda fuera de esta funcionalidad.
- **Gating:** no hace falta nada nuevo, porque la pizarra ya está detrás de `runIfPremium` en el Home. Es una mejora de algo que ya es premium, más que un argumento de venta propio.
- Riesgo bajo: es solo interfaz y no toca el formato guardado de `Play`.

8. Backup en la nube (equipos y jugadas)

Alto | Carga en vivo: no | Backend: reglas nuevas (y Storage si hay fotos) | Datos salen del dispositivo: sí

Objetivo. Que un entrenador no pierda nada si cambia de dispositivo o reinstala. **Con el sync (3) hecho, este punto se achica bastante:** los partidos terminados ya quedan por cuenta, y lo que falta es todo lo demás.

Dato	¿Cubierto?
Partidos terminados	✓ por el sync (3)
Partidos en curso	No se suben (ver 3)
Equipos y planteles (`teams_box`)	✗ falta
Jugadas de pizarra (`plays_box`)	✗ falta
Fotos de jugadores (archivos en `player_photos/`)	✗ requiere Storage
Fixture (si se hace el 6)	✗ falta
Preferencias (tema, gráficos)	No hace falta



Boceto. Qué queda cubierto con el sync y qué agrega el backup, más una entrada manual en Configuración (`account_settings_screen.dart`).

Dos caminos posibles

A · EXTENDER EL SYNC A EQUIPOS Y JUGADAS

- Misma mecánica que el 3: `teams/{uid}/…` y `plays/{uid}/…`, con push al guardar y pull al abrir.
- Diferencia clave: un equipo **se edita** muchas veces, a diferencia de un partido terminado. Hace falta un `updatedAt` y una regla de "gana la última edición".
- Más transparente para el usuario y más trabajo de pruebas.

B · BACKUP MANUAL (RECOMENDADO PARA EMPEZAR)

- Botones "Hacer backup" y "Restaurar" que suben o bajan una foto completa de equipos y jugadas.
- Sin conflictos: restaurar reemplaza o combina por id, a elección del usuario.
- Ojo con el límite de 1 MiB por documento si hay muchas jugadas: en ese caso conviene un archivo JSON en Storage.

Riesgos

- **Fotos:** `Player.photoPath` es una ruta local. Si se restaura en otro dispositivo, esa ruta no existe y `FileImage` falla en `team_form_screen.dart`. Recomendación para la primera versión: no incluir fotos y limpiar `photoPath` al restaurar.
- **Privacidad:** los planteles incluyen nombre y fecha de nacimiento, posiblemente de menores. Va en la misma actualización de la política de privacidad que pide el sync.
- **Costo:** Storage y Firestore cobran a partir de cierto volumen; con backup manual el costo es acotado y predecible.

9. Scouting fase 2 y carga precisa (Etapa 3)

Alto | Carga en vivo: sí | Backend: no | Datos salen del dispositivo: no

Objetivo. Dos mejoras de la misma naturaleza, porque las dos **agregan datos que hoy no se capturan durante el partido**:

- **Scouting fase 2:** registrar a qué zona ataca el *rival* para armar su mapa de calor en el scouting. Hoy `computeZones()` y los mapas solo cubren al equipo propio.
- **Etapa 3 de la estadística visual** (pospuesta el 02/10): tocar el punto exacto de la cancha (`targetX`/`targetY`, origen opcional) en vez de elegir una zona, según `spec-estadistica-visual.md` §6.



Boceto. Paso opcional en el diálogo de punto del rival (con "Omitir" para no frenar la carga) y el mapa de calor resultante en el scouting.

QUÉ HAY QUE CONSTRUIR

- `rally_event.dart`: `rivalTargetZone` (fase 2) y/o `targetX/Y`, `originX/Y` (Etapa 3), opcionales y `null` en partidos viejos.
- `rival_action_dialog.dart` / `hit_zone_picker.dart` / `touch_dialog.dart`: el selector en vivo.
- `match_controller.dart` (`_addEvent`): guardar el dato nuevo.
- `scouting_engine.dart`: agregar zonas por rival y mostrarlas con `zone_heatmap.dart`.

POR QUÉ ES ALTO

- Toca el **corazón de la app** y el formato guardado de `RallyEvent`; requiere probar con partidos completos.
- Cada toque extra durante un partido en vivo compite con la velocidad de carga, que es la principal virtud de la app. El paso tiene que ser opcional y rápido.
- El valor llega con retraso: solo los partidos cargados *después* del release tienen el dato nuevo.

> **Recomendación:** retomarlo después de validar con usuarios reales las Etapas 1 y 2 (todavía sin publicar). Si se hace, conviene hacer fase 2 y Etapa 3 juntas, porque tocan los mismos archivos de la carga en vivo y conviene hacer una sola ronda de pruebas.

10. Chatbot conversacional (IA)

Muy alto | Carga en vivo: no | Backend: Cloud Function nueva | Datos salen del dispositivo: sí (anonimizados)

Objetivo. Hacer preguntas en lenguaje natural sobre el partido abierto ("¿quién recibió mejor en el set 3?", "¿por qué no me deja un tercer cambio de líbero?") y recibir respuestas redactadas a partir de datos que la app ya calculó. No hace falta entrenar un modelo propio: se usa un modelo existente vía API y se le pasa el contexto en cada consulta.



Diagrama y boceto. Versión acotada recomendada en el roadmap (opción A): preguntas sobre el partido abierto y sobre el manual, con nombres anonimizados y un tope mensual.

QUÉ YA EXISTE

- En el backend, `redeemPromoCode.ts` ya es un `onCall` con secreto (`defineSecret`) y verificación de entitlements: sirve de molde para el proxy.
- `StatsEngine` y `visual_stats.dart` dan el contexto ya calculado.
- `tool/generate_manual.dart` : el texto del manual para las preguntas de reglas y de uso.

QUÉ HAY QUE CONSTRUIR

- `functions/src/askCoach.ts` : auth, premium, cupo, llamada a la API y registro de uso.
- Cliente: el armado del contexto con anonimización y re-mapeo de nombres.
- `lib/screens/chatbot/` : pantalla con historial, estados de carga y error, y mensaje sin conexión.
- Política de privacidad, ficha de Play y tope de costo.

Por qué es el más grande: es la única con **costo variable por uso**, suma una tercera pieza de infraestructura (además de Firebase y RevenueCat) y obliga a acordar antes cómo manejar alucinaciones y datos de menores.

C. Orden sugerido de implementación

No es un compromiso, es una propuesta según esfuerzo, riesgo y dependencias entre funcionalidades.

Tanda 1 Hecha

Implementada el 04/10/2026 · versión 1.2.0

- **1** Notas de scouting
- **2** Exportar a CSV
- **4** Comparador de 2 partidos

Nada toca la carga en vivo ni el backend. Deja hecha la tabla compartida que usa la tanda 2.

Tanda 2

Reporte multi-partido + sync

- **5** Panel de temporada
- **3** Etapa 1: datos locales por cuenta, *puede ir antes*
- **3** Etapa 2: sync, *en una versión aparte*

La etapa 1 arregla un bug actual. La 2 necesita la prueba de actualizar encima, dos dispositivos y la política de privacidad nueva.

Tanda 3

Organización y pizarra

- **6** Fixture
- **7** Animar jugada de pizarra

Antes de arrancar: decidir si el fixture es gratis y si entra en el sync.

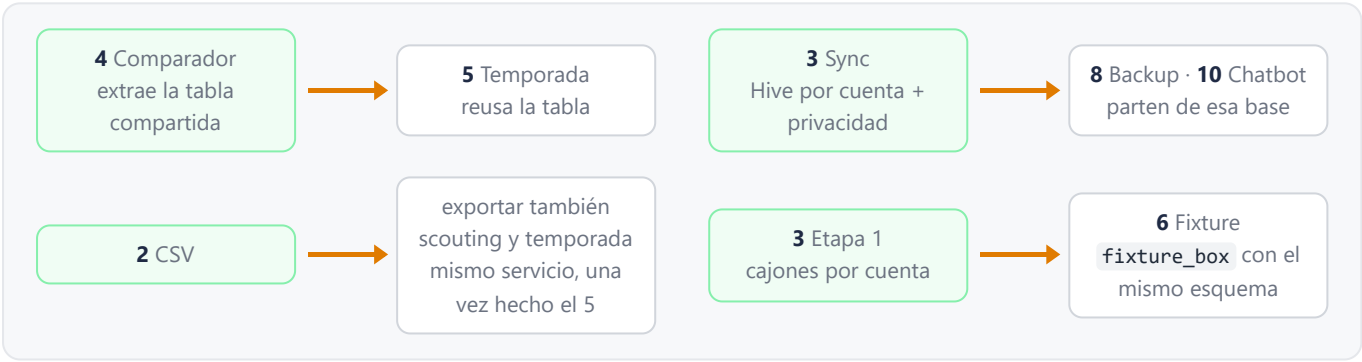
Más adelante

Conversar el alcance primero

- **8** Backup (más chico después del 3)
- **9** Scouting fase 2 / Etapa 3
- **10** Chatbot IA

Dependen de feedback real sobre las Etapas 1 y 2 y de definir costo y política de datos.

Dependencias entre funcionalidades



Decisiones de producto que conviene cerrar

Decisión	Afecta a	Sugerencia
¿El scouting de rivales (ya hecho) pasa a ser premium?	Scouting, Notas (1)	Hoy está libre. Si se gatea, las notas suman valor del lado pago.
¿El fixture es gratis, premium o tiene un tope?	Fixture (6)	Gratis o tope de 5 filas: es un gancho de uso frecuente.
¿La nube de partidos es gratis o premium?	Sync (3)	Gratis: es el respaldo de lo que el usuario ya tiene. El valor pago es el archivo ilimitado y los dispositivos simultáneos.
¿Qué datos se sincronizan además de los partidos?	Sync (3), Fixture (6), Backup (8)	Empezar solo con partidos y decidir el resto con el sync andando.
Texto nuevo de la política de privacidad y de la ficha de Play	Sync (3), Backup (8), Chatbot (10)	Redactarlo una sola vez, junto con el sync, contemplando datos de menores.